无线广域网数据传输仿真

本章主要介绍蜂窝网络通信的发展，探讨 4G、5G 特点和未来 6G 发展情况。相比 5G 网络，4G 网络的覆盖范围更大，因而实验重点利用 NS-3 分析 4G 网络的数据传输性能，特别是基站切换对数据传输的影响，旨在让读者掌握蜂窝网络传输性能分析流程。

7.1 预备知识

7.1.1 蜂窝通信发展

蜂窝网络通信，尤其是 5G/6G 网络，正经历快速的发展和变革，以下是蜂窝网络通信技术发展的主要历程：

第 1 至 4 代蜂窝通信技术（1G-4G）：20 世纪 70 年代末至 80 年代初推出 1G 网络，采用模拟信号通信，可提供语音通话服务。由于模拟系统存在频道数量有限、易受干扰、无法加密等问题，1990 年初推出 2G 网络，使用数字信号处理技术提高了通信质量和安全性，并引入短信服务。为提供低速数据传输服务，在 2G 基础上引入 GPRS 和 EDGE，形成 2.5G 网络。在 2000 年初，提供更高数据速率的 3G 网络商用，支持图像、音乐、视频等媒体内容处理，使移动互联网服务变得更加普及。在 3G 基础上引入 HSPA 和 HSPA+（支持更快下载速度），进一步提高数据速率，形成了 3.5G。自 2010 年开始，4G 开始商用，可提供极高数据速率、低延迟，支持高清视频流、视频通话和高速互联网接入。4G 技术进一步推动了移动互联网和移动应用的快速发展，本章后续将深入探讨。

第五代蜂窝通信技术（5G）：从 2019 年开始逐步商用，目标是提供更高数据传输速率（理论峰值下载速率可达数十 Gbps，用户体验速率预计达 1Gbps 以上）、更低时延（1 毫秒以下的通信延迟）、更大设备连接能力（每平方公里可同时连接多达 100 万台设备）、更高可靠性和更广泛的应用服务等。5G 支持增强移动宽带（eMBB，适用于高清视频流、虚拟现实和增强现实等）、低延迟高可靠通信（uRLLC，适用于自动驾驶、工业自动化、远程医疗等）、大规模机器类型通信（mMTC，适用于智能城市、智能农业、智能家居等）。5G 采用了一系列新技术，如大规模 MIMO、毫米波通信、网络切片等。

未来第六代蜂窝通信技术（6G）：目前尚处于研究和开发阶段。6G 目标是在 5G 基础上提供更高的数据传输速率（预计峰值速率达到 1Tbps 或更高）、更低的延迟（亚毫秒甚至微秒通信延迟）、更大的连接容量（支持更多设备同时连接）、更广泛的频谱（如太赫兹（THz）频段）、

更高的能源效率、更智能的网络（采用机器学习实现网络自我优化、自我修复和智能资源分配）和更广泛的应用场景（例如，全息通信，实现高清晰度 3D 全息视频通话和远程呈现；远程感知和操控，提供高精度、实时的远程感知和操控能力；智能城市和基础设施，支持更加智能化的城市管理和基础设施监控；增强现实和虚拟现实，提供无缝的增强现实和虚拟现实体验）。

蜂窝通信技术的发展是不断提高网络速度、扩大覆盖范围、降低成本、增强网络容量和可靠性，同时支持更多样化的服务和应用。随着技术不断进步，未来蜂窝通信网络将更加智能、高效，为用户提供更加丰富和便捷的通信体验。

7.1.2 4G LTE 蜂窝通信

在 4G 网络中，LTE 是主要技术标准之一[1]，由 3GPP 组织制定，具备全 IP 网络、网络扁平化、MIMO、OFDMA 等特点，如图 7.1 所示。LTE 是 3G 技术的演进，旨在提供更高的数据传输速率（支持高达 100Mbps 的下行速率和 50Mbps 的上行速率，实际速率受网络负载和地理位置等因素影响）、更低延迟（低于 5 毫秒的通信延迟）、更宽频谱带宽（1.4MHz 至 20MHz 可变带宽）。从传输速率上看，LTE 只能算 3.9G，只有升级版的 LTE Advanced 才满足国际电信联盟对 4G 的要求。

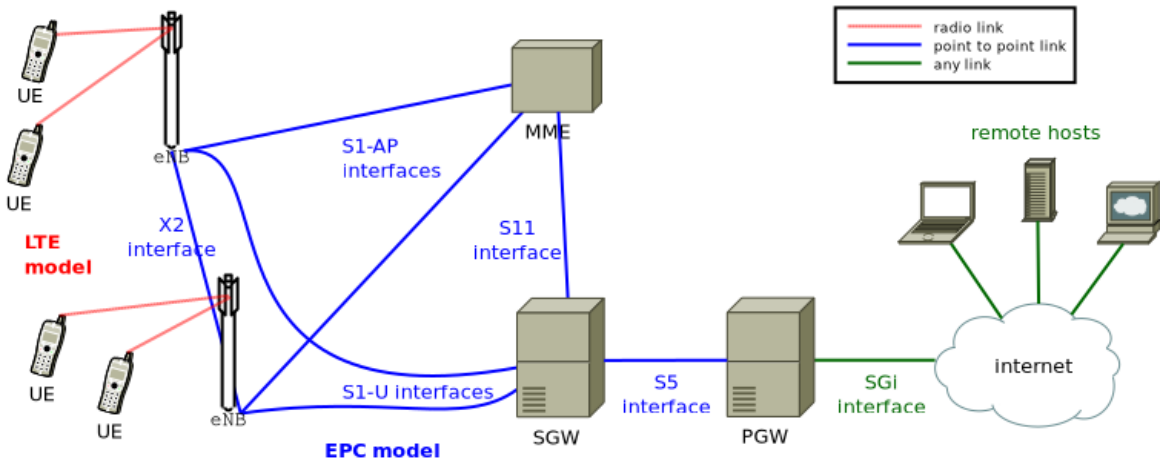


图 7.1　LTE 基本构成示意图

为满足 4G 性能要求，LTE 提供了 4 部分关键组件，具体包括：①用户设备（User Equipment, UE），如智能手机，通过射频信道与基站进行通信；②基站（Base Station, BS），4G 中称为 **Evolved Node B**（eNodeB），而 5G 中称为 **the next Generation Node B**（gNodeB），连接 UE 并与核心网进行通信，负责无线接入和数据传输；③核心网（Core Network, CN），实现用户数据传输/鉴权/计费等功能，由多个网络元素构成，如移动管理实体（Mobility Management Entity, MME）、

[1] https://www.20nn.cn/tongxin/net-lte.html

服务网关（Serving Gateway, SGW）和数据包网络网关（Packet Data Network Gateway, PGW）等；④公共数据网络（Public Data Network, PDN），提供互联网连接和其他服务，如 IP 网络和云服务等。LTE 的具体特性如下：

①被称为演进的 UMTS 陆地无线接入网（E-UTRAN），如图 7.2 所示，简化了 3G 中 RNC（无线网络控制器）和 BSC（基站控制器）等节点，仅保留 eNodeB；

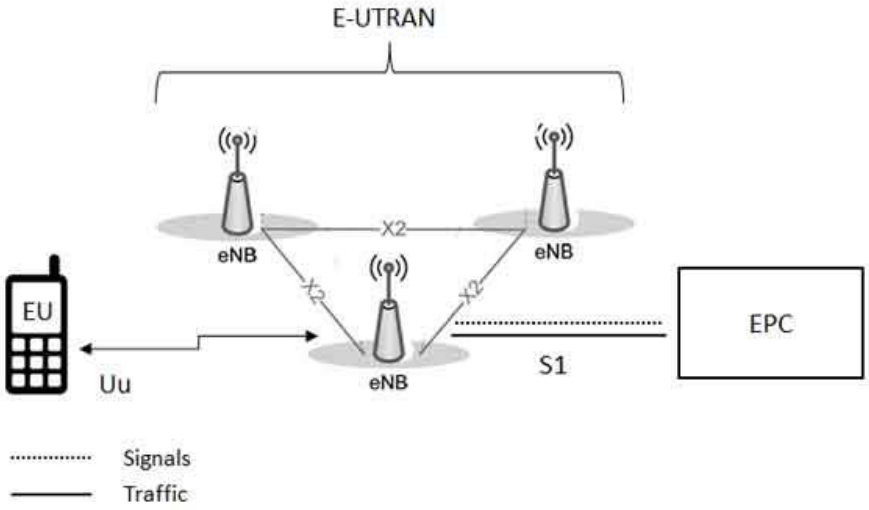


图 7.2　E-UTRAN 示意图

②演进分组核心（Evolved Packet Core, EPC）由 MME、SGW、PGW 等网元构成，可支持 3GPP 和非 3GPP（如 Wi-Fi、WiMAX 等）多种接入方式的异构网络融合架构；

③支持更精细 QoS（服务质量）管理，确保不同类型业务能获得所需网络资源；

④支持通过 IP 网络进行语音通话，即 Voice over LTE（VoLTE），提供了比传统电路交换语音更高的语音质量。

为使网络业务更具灵活性，EPC 核心网将用户数据和信令分离，即分成用户平面和控制平面。用户平面负责业务数据的传送和处理，控制平面负责协调和控制信令的传送和处理。

7.2 实验环境

在本章实验中，主要分析 TCP 在 4G LTE 网络中基站切换时的性能。实验所使用的仿真器为 NS-3（版本 3.29），其他版本的仿真器需自行移植，操作系统为运行于 **VirtualBox 6.1.50** 中的 **Ubuntu 20.04.1**。对于 5G 网络场景的仿真，可参考 NYU Wireless[2]研究中心的 mmWave 项目[3]，在本章中不进行探究。

在 LTE-EPC 数据平面协议栈中（如图 7.3 所示），NS-3 中实现了图 7.3 所示的数据路径上

[2] https://wireless.engineering.nyu.edu/
[3] https://github.com/nyuwireless-unipd/ns3-mmwave

的所有节点（有关 NS-3 中 LTE 实现可参考官方文档[4]），即 UE、eNB、SGW、PGW 和 Internet 中的远程主机，支持 3GPP 指定的所有协议栈(S5 协议栈、S1-U 协议栈和 LTE 无线协议栈)。

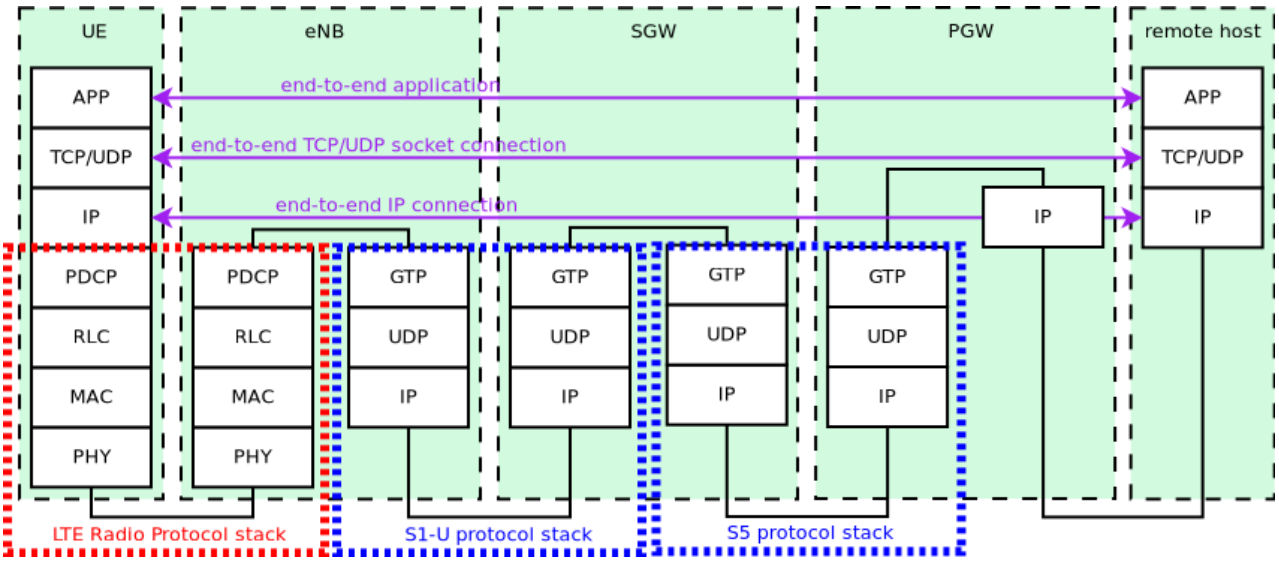


图 7.3　LTE-EPC 数据平面协议栈

LTE-EPC 控制平面协议栈如图 7.4 所示，NS-3 显式建模的控制接口包括 S1-MME、S11、S5 接口和 X2 接口。S1-MME、S11 和 S5 接口使用 SCTP 协议作为传输协议，利用其各自链路上发送的协议数据单元进行建模。在 LTE-EPC 建模时，NS-3 中尚未实现 SCTP 协议，因而使用 UDP 代替 SCTP。

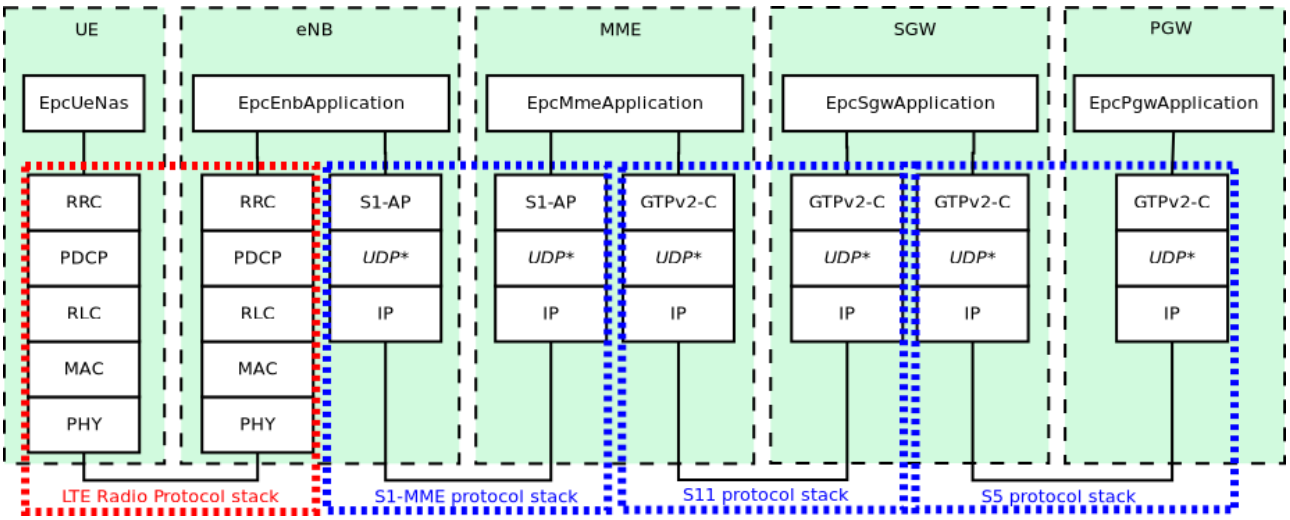


图 7.4　LTE-EPC 控制面协议栈

7.3 实验步骤

为便于实验过程描述，本章实验的根路径设置为“**/home/wiotlab/WIoTLab/tools/ns3**”，

[4] https://www.nsnam.org/docs/models/html/lte-design.html

ns3 为实验所需的源码[5]目录，其中包含了 bake、netanim-3.108、ns-3.29 等文件夹和 build.py、constants.py、util.py 等文件。

7.3.1 NS-3 源码编译

在正式编译 NS-3 前，需安装其编译所需的依赖软件，具体可参考官方 Wiki[6]。为便于快速开展本章实验，此处给出了如下所示的完整命令脚本。读者可将脚本保存为“**install_dep.sh**”文件并放置在 Ubuntu 20.04.1 的用户目录中，执行“**chmod +x install_dep.sh; ./install_dep.sh**”进行快速安装。安装过程中可能会提示输入系统密码。

```
sudo apt-get update
sudo apt-get -y install build-essential libsqlite3-dev libboost-all-dev libssl-dev git python3-setuptools castxml gir1.2-goocanvas-2.0 gir1.2-gtk-3.0 libgirepository1.0-dev python3-dev python3-gi python3-gi-cairo python3-pip python3-pygraphviz python3-pygccxml
sudo apt-get -y install g++ pkg-config sqlite3 qt5-default mercurial ipython3 openmpi-bin openmpi-common openmpi-doc libopenmpi-dev autoconf cvs bzr unrar gdb valgrind uncrustify doxygen graphviz imagemagick python3-sphinx dia tcpdump libxml2 libxml2-dev cmake libc6-dev libc6-dev-i386 libclang-6.0-dev llvm-6.0-dev automake vtun lxc uml-utilities libgtk-3-dev
sudo ln -s /usr/bin/python3 /usr/bin/python
python -m pip install --user cxxfilt pybindgen
```

若在安装 NS-3 依赖软件过程中出现无法找到软件的问题，可将下述内容添加到“**/etc/apt/sources.list**”文件中，并执行 **sudo apt update** 进行更新。

```
deb http://archive.ubuntu.com/ubuntu/ trusty main universe restricted multiverse
deb https://mirrors.aliyun.com/ubuntu/ xenial main
deb-src https://mirrors.aliyun.com/ubuntu/ xenial main
deb https://mirrors.aliyun.com/ubuntu/ xenial universe
deb-src https://mirrors.aliyun.com/ubuntu/ xenial universe
deb http://cz.archive.ubuntu.com/ubuntu bionic main universe
deb http://mirrors.aliyun.com/ubuntu bionic main universe
```

在依赖软件成功安装后，就可开始编译 NS-3。使用 **cd** 命令将当前工作目录切换至“**/home/wiotlab/WIoTLab/tools/ns3**”，然后执行“**./build.py --enable-examples --enable-tests**”进行 NS-3 编译。编译成功后，会出现图 7.5 所示的结果输出。

由于本章实验采用了华盛顿大学 FUNLAB 在其 EE595 课程中修改的 NS-3（版本 3.29）源码[7]，因而需要对其进行编译。下载源码放置于“**/home/wiotlab/WIoTLab/tools/ns3**”文件夹中并命名为“**handover**”。使用 **cd** 命令切换到“**handover**”文件夹中，执行“**./waf configure --enable-examples -d optimized; ./waf build**”。若提示 **waf** 找不到或权限问题，请执行“**chmod**

[5] https://www.nsnam.org/releases/ns-allinone-3.29.tar.bz2
[6] https://www.nsnam.org/wiki/Installation
[7] https://gitlab.com/tomhenderson/ns-3-ee-595

+x waf”增加权限。

```
Modules built:
antenna                  aodv                       applications
bridge                   buildings                  config-store
core                     csma                       csma-layout
dsdv                     dsr                        energy
fd-net-device            flow-monitor               internet
internet-apps            lr-wpan                    lte
mesh                     mobility                   mpi
netanim (no Python)      network                    nix-vector-routing
olsr                     point-to-point             point-to-point-layout
propagation              simple-wireless (no Python) sixlowpan
spectrum                 stats                      tap-bridge
test (no Python)         topology-read              traffic-control
uan                      virtual-net-device         wave
wifi                     wimax

Modules not built (see ns-3 tutorial for explanation):
brite                    click                      openflow
```

图 7.5　NS-3 编译成功的输出

7.3.2 实验代码解析

实验构建了如图 7.6 所示的网络拓扑，其中拓扑范围（x2Distance 和 yDistanceForUe）、UE 运动速度、基站切换方式等可通过命令行修改。文件服务器传输的文件大小为 200MB。

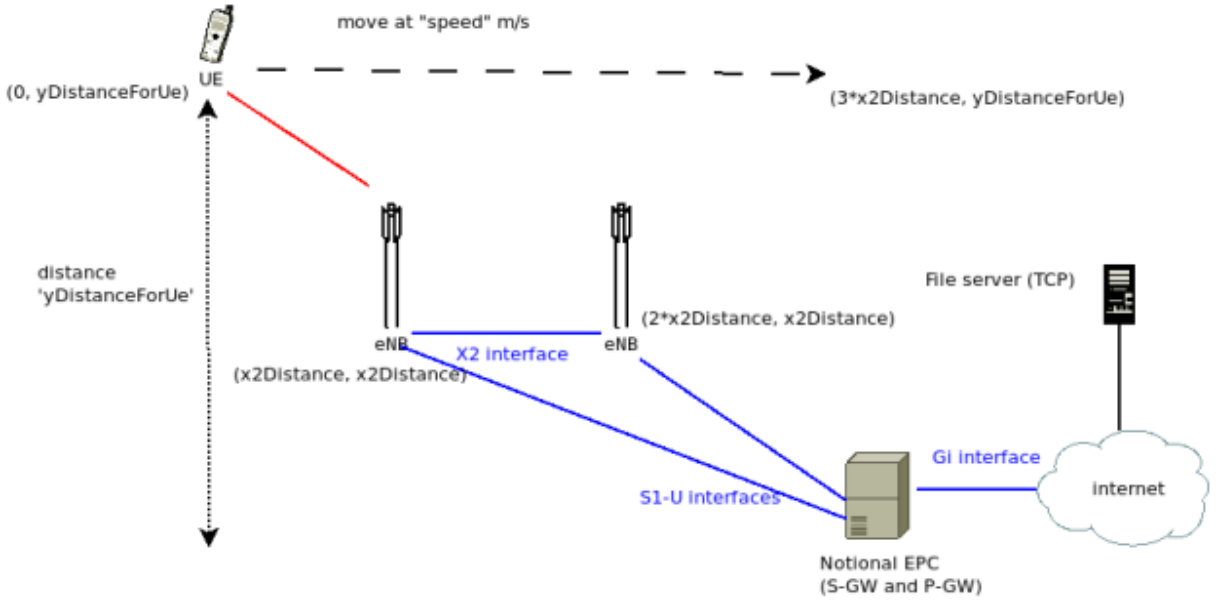


图 7.6　实验拓扑示意

实验的完整代码如下所示：

```
#include <iostream>
#include <iomanip>
#include <fstream>
#include <string>
#include "ns3/core-module.h"
#include "ns3/network-module.h"
```

```
#include "ns3/internet-module.h"
#include "ns3/mobility-module.h"
#include "ns3/lte-module.h"
#include "ns3/applications-module.h"
#include "ns3/point-to-point-module.h"
using namespace ns3;
NS_LOG_COMPONENT_DEFINE ("LteTcpX2Handover");
std::ofstream g_ueMeasurements;
std::ofstream g_packetSinkRx;
std::ofstream g_cqiTrace;
std::ofstream g_tcpCongStateTrace;
std::ofstream g_positionTrace;
//执行进度报告函数
void ReportProgress (Time reportingInterval) {
    std::cout << "*** Simulation time: " << std::fixed << std::setprecision (1) << Simulator::Now ().GetSeconds () << "s" << std::endl;
    Simulator::Schedule (reportingInterval, &ReportProgress, reportingInterval);
}
//从类似”/NodeList/3/DeviceList/1/Mac/Assoc”的字符串中抽取节点 ID
uint32_t ContextToNodeId (std::string context) {
    std::string sub = context.substr (10);
    uint32_t pos = sub.find ("/Device");
    return atoi (sub.substr (0,pos).c_str ());
}
void NotifyUdpReceived (std::string context, Ptr<const Packet> p, const Address &addr) {
    std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
    << " UDP received" << std::endl;
}
void NotifyConnectionEstablishedUe (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti) {
    std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
    << " UE IMSI " << imsi<< ": connected to CellId " << cellid<< " with RNTI " <<rnti<< std::endl;
}
void NotifyHandoverStartUe (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti, uint16_t targetCellId) {
    std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
    << " UE IMSI " << imsi<< ": previously connected to CellId " << cellid << " with RNTI " << rnti
    << ", doing handover to CellId " << targetCellId<< std::endl;
}
void NotifyHandoverEndOkUe (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti) {
    std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
    << " UE IMSI " << imsi<< ": successful handover to CellId " << cellid
    << " with RNTI " << rnti<< std::endl;
}
void NotifyConnectionEstablishedEnb (std::string context,uint64_t imsi,uint16_t cellid,uint16_t rnti) {
```

```
	std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
	<< " eNB CellId " << cellid<< ": successful connection of UE with IMSI " << imsi
	<< " RNTI " << rnti<< std::endl;
}

void NotifyHandoverStartEnb (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti, uint16_t targetCellId) {
	std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
	<< " eNB CellId " << cellid<< ": start handover of UE with IMSI " << imsi
	<< " RNTI " << rnti<< " to CellId " << targetCellId<< std::endl;
}
void NotifyHandoverEndOkEnb (std::string context,uint64_t imsi,uint16_t cellid,uint16_t rnti) {
	std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
	<< " eNB CellId " << cellid<< ": completed handover of UE with IMSI " << imsi
	<< " RNTI " << rnti<< std::endl;
}
void TracePosition (Ptr<Node> ue, Time interval) {
	Vector v = ue->GetObject<MobilityModel> ()->GetPosition ();
	g_positionTrace << std::setw (7) << std::setprecision (3) << std::fixed
	<< Simulator::Now ().GetSeconds () << " "<< v.x << " " << v.y << std::endl;
	Simulator::Schedule (interval, &TracePosition, ue, interval);
}
void NotifyUeMeasurements (std::string context, uint16_t rnti, uint16_t cellId, double rsrpDbm, double rsrqDbm, bool servingCell, uint8_t ccId) {
	g_ueMeasurements << std::setw (7) << std::setprecision (3) << std::fixed
	<< Simulator::Now ().GetSeconds () << " " << std::setw (3) << cellId << " "
	<< std::setw (3) << (servingCell ? "1" : "0") << " " << std::setw (8) << rsrpDbm
	<< " " << std::setw (8) << rsrqDbm << std::endl;
}
void NotifyPacketSinkRx (std::string context, Ptr<const Packet> packet, const Address &address) {
	g_packetSinkRx << std::setw (7) << std::setprecision (3) << std::fixed
	<< Simulator::Now ().GetSeconds () << " " << std::setw (5) << packet->GetSize () << std::endl;
}
void NotifyCqiReport (std::string context, uint16_t rnti, uint8_t cqi) {
	g_cqiTrace << std::setw (7) << std::setprecision (3) << std::fixed
	<< Simulator::Now ().GetSeconds () << " " << std::setw (4) << ContextToNodeId (context)
	<< " " << std::setw (4) << rnti    << " " << std::setw (3) << static_cast<uint16_t> (cqi) << std::endl;
}
void CongStateTrace (const TcpSocketState::TcpCongState_t oldValue, const TcpSocketState::TcpCongState_t newValue) {
	g_tcpCongStateTrace << std::setw (7) << std::setprecision (3) << std::fixed
	<< Simulator::Now ().GetSeconds () << " "<< std::setw (4)
	<< TcpSocketState::TcpCongStateName[newValue] << std::endl;
}
```

```
void ConnectTcpTrace (void) {
    Config::ConnectWithoutContext ("/NodeList/1/$ns3::TcpL4Protocol/SocketList/0/CongState",
MakeCallback (&CongStateTrace));
}
//基于 RSRQ 度量的 X2 自动切换程序，其实例化两个 eNodeB，UE 关联到其中一个 eNodeB
//在 UE 移动过程中，其测量 RSRQ 并进行切换
int main (int argc, char *argv[]) {
    uint16_t numberOfUes = 1;
    uint16_t numberOfEnbs = 2;
    //命令行可改变的参数
    double x2Distance = 500.0; // m
    double yDistanceForUe = 1000.0;    // m
    double speed = 20;            // m/s
    double enbTxPowerDbm = 46.0;
    std::string handoverType = "A2A4";
    bool useRlcUm = false;
    bool verbose = false;
    bool pcap = false;
    //其他常量
    LogLevel logLevel = (LogLevel)(LOG_PREFIX_ALL | LOG_LEVEL_ALL);
    std::string traceFilePrefix = "lte-tcp-x2-handover";
    Time positionTracingInterval = Seconds (5);
    Time reportingInterval = Seconds (10);
    uint32_t ftpSize = 200000000; // 200 MB
    uint16_t port = 4000;   //端口号
    Config::SetDefault ("ns3::LteHelper::UseIdealRrc", BooleanValue (true));
    CommandLine cmd;
    cmd.AddValue ("speed", "Speed of the UE (m/s)", speed);
    cmd.AddValue ("x2Distance", "Distance between eNB at X2 (meters)", x2Distance);
    cmd.AddValue ("yDistanceForUe", "y value (meters) for UE", yDistanceForUe);
    cmd.AddValue ("enbTxPowerDbm", "TX power (dBm) used by eNBs", enbTxPowerDbm);
    cmd.AddValue ("useRlcUm", "Use LTE RLC UM mode", useRlcUm);
    cmd.AddValue ("handoverType", "Handover type (A2A4 or A3Rsrp)", handoverType);
    cmd.AddValue ("pcap", "Enable pcap tracing", pcap);
    cmd.AddValue ("verbose", "Enable verbose logging", verbose);
    cmd.Parse (argc, argv);
    double simTime = 50; //仿真时长
    //  如果移动开启，根据速度调整仿真时长
    if (speed < 10 && speed != 0) {
        std::cout << "Select a speed at least 10 m/s, or zero" << std::endl;
        exit (1);
    } else if (speed >= 10) {
        // Handover around the middle of the total simTime
        simTime = (double)(numberOfEnbs + 1) * x2Distance / speed;
```

```
    }
    if (verbose) {
        LogComponentEnable ("EpcX2", logLevel);
        LogComponentEnable ("A2A4RsrqHandoverAlgorithm", logLevel);
        LogComponentEnable ("A3RsrpHandoverAlgorithm", logLevel);
    }
    if (useRlcUm == false) {
        Config::SetDefault ("ns3::LteEnbRrc::EpsBearerToRlcMapping", EnumValue
(LteEnbRrc::RLC_AM_ALWAYS));
    }
    g_ueMeasurements.open ((traceFilePrefix + ".ue-measurements.dat").c_str(), std::ofstream::out);
    g_ueMeasurements << "# time  cellId  isServingCell?  RSRP(dBm)  RSRQ(dB)" << std::endl;
    g_packetSinkRx.open ((traceFilePrefix + ".tcp-receive.dat").c_str(), std::ofstream::out);
    g_packetSinkRx << "# time  bytesRx" << std::endl;
    g_cqiTrace.open ((traceFilePrefix + ".cqi.dat").c_str(), std::ofstream::out);
    g_cqiTrace << "# time  nodeId  rnti  cqi" << std::endl;
    g_tcpCongStateTrace.open ((traceFilePrefix + ".tcp-state.dat").c_str(), std::ofstream::out);
    g_tcpCongStateTrace << "# time  congState" << std::endl;
    g_positionTrace.open ((traceFilePrefix + ".position.dat").c_str(), std::ofstream::out);
    g_positionTrace << "# time  congState" << std::endl;
    Ptr<LteHelper> lteHelper = CreateObject<LteHelper> ();
    Ptr<PointToPointEpcHelper> epcHelper = CreateObject<PointToPointEpcHelper> ();
    lteHelper->SetEpcHelper (epcHelper);
    lteHelper->SetSchedulerType ("ns3::RrFfMacScheduler");
    if (handoverType == "A2A4") {
        lteHelper->SetHandoverAlgorithmType ("ns3::A2A4RsrqHandoverAlgorithm");
        lteHelper->SetHandoverAlgorithmAttribute ("ServingCellThreshold", UintegerValue (30));
        lteHelper->SetHandoverAlgorithmAttribute ("NeighbourCellOffset", UintegerValue (1));
    } else if (handoverType == "A3Rsrp") {
        lteHelper->SetHandoverAlgorithmType ("ns3::A3RsrpHandoverAlgorithm");
        lteHelper->SetHandoverAlgorithmAttribute ("Hysteresis", DoubleValue (3.0));
        lteHelper->SetHandoverAlgorithmAttribute ("TimeToTrigger", TimeValue (MilliSeconds (256)));
    } else {
        std::cout << "Unknown handover type: " << handoverType << std::endl;
        exit (1);
    }
    //创建单个 RemoteHost
    NodeContainer remoteHostContainer;
    remoteHostContainer.Create (1);
    Ptr<Node> remoteHost = remoteHostContainer.Get (0);
    InternetStackHelper internet;
    internet.Install (remoteHostContainer);
    // 创建假象 Internet
    PointToPointHelper p2ph;
```

```
p2ph.SetDeviceAttribute ("DataRate", DataRateValue (DataRate ("100Gb/s")));
p2ph.SetDeviceAttribute ("Mtu", UintegerValue (1500));
p2ph.SetChannelAttribute ("Delay", TimeValue (Seconds (0.010)));
NetDeviceContainer internetDevices = p2ph.Install (epcHelper->GetPgwNode (), remoteHost);
Ipv4AddressHelper ipv4h;
ipv4h.SetBase ("1.0.0.0", "255.0.0.0");
Ipv4InterfaceContainer internetIpIfaces = ipv4h.Assign (internetDevices);
Ipv4StaticRoutingHelper ipv4RoutingHelper;
Ptr<Ipv4StaticRouting> remoteHostStaticRouting =
        ipv4RoutingHelper.GetStaticRouting (remoteHost->GetObject<Ipv4> ());
//接口 0 为 localhost，接口 1 为 p2p 设备
remoteHostStaticRouting->AddNetworkRouteTo (Ipv4Address ("7.0.0.0"),
                                            Ipv4Mask ("255.0.0.0"), 1);
NodeContainer ueNodes;
NodeContainer enbNodes;
enbNodes.Create (numberOfEnbs);
ueNodes.Create (numberOfUes);
//eNB 上安装移动模型
Ptr<ListPositionAllocator> enbPositionAlloc = CreateObject<ListPositionAllocator> ();
for (uint16_t i = 0; i < numberOfEnbs; i++){
    Vector enbPosition (x2Distance * (i + 1), x2Distance, 0);
    enbPositionAlloc->Add (enbPosition);
}
MobilityHelper enbMobility;
enbMobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");
enbMobility.SetPositionAllocator (enbPositionAlloc);
enbMobility.Install (enbNodes);
//UE 上安装移动模型
MobilityHelper ueMobility;
ueMobility.SetMobilityModel ("ns3::ConstantVelocityMobilityModel");
ueMobility.Install (ueNodes);
ueNodes.Get (0)->GetObject<MobilityModel> ()->SetPosition (Vector (0, yDistanceForUe, 0));
ueNodes.Get (0)->GetObject<ConstantVelocityMobilityModel> ()
                ->SetVelocity (Vector (speed, 0, 0));
//在 eNB 和 UE 上安装 LTE 设备
Config::SetDefault ("ns3::LteEnbPhy::TxPower", DoubleValue (enbTxPowerDbm));
NetDeviceContainer enbLteDevs = lteHelper->InstallEnbDevice (enbNodes);
NetDeviceContainer ueLteDevs = lteHelper->InstallUeDevice (ueNodes);
//UE 上安装 IP 栈
internet.Install (ueNodes);
Ipv4InterfaceContainer ueIpIfaces;
ueIpIfaces = epcHelper->AssignUeIpv4Address (NetDeviceContainer (ueLteDevs));
//将所有 UE 关联至第 1 个 eNodeB
for (uint16_t i = 0; i < numberOfUes; i++){
```

```
        lteHelper->Attach (ueLteDevs.Get (i), enbLteDevs.Get (0));
    }
    Ptr<Ipv4StaticRouting> ueStaticRouting =
                ipv4RoutingHelper.GetStaticRouting (ueNodes.Get (0)->GetObject<Ipv4> ());
    ueStaticRouting->SetDefaultRoute (epcHelper->GetUeDefaultGatewayAddress (), 1);
    BulkSendHelper ftpServer ("ns3::TcpSocketFactory", Address ());
    AddressValue remoteAddress (InetSocketAddress (ueIpIfaces.GetAddress (0), port));
    ftpServer.SetAttribute ("Remote", remoteAddress);
    ftpServer.SetAttribute ("MaxBytes", UintegerValue (ftpSize));
    NS_LOG_LOGIC ("setting up TCP flow from remote host to UE");
    ApplicationContainer sourceApp = ftpServer.Install (remoteHost);
    sourceApp.Start (Seconds (1));
    sourceApp.Stop (Seconds (simTime));
    Address sinkLocalAddress (InetSocketAddress (Ipv4Address::GetAny (), port));
    PacketSinkHelper sinkHelper ("ns3::TcpSocketFactory", sinkLocalAddress);
    ApplicationContainer sinkApp = sinkHelper.Install (ueNodes.Get (0));
    sinkApp.Start (Seconds (1));
    sinkApp.Stop (Seconds (simTime));
    Ptr<EpcTft> tft = Create<EpcTft> ();
    EpcTft::PacketFilter dlpf;
    dlpf.localPortStart = port;
    dlpf.localPortEnd = port;
    tft->Add (dlpf);
    EpsBearer bearer (EpsBearer::NGBR_VIDEO_TCP_DEFAULT);
    lteHelper->ActivateDedicatedEpsBearer (ueLteDevs.Get (0), bearer, tft);
    //添加 X2 接口
    lteHelper->AddX2Interface (enbNodes);
    //追踪数据包
    if (pcap){
        p2ph.EnablePcapAll ("lte-tcp-x2-handover");
    }
    lteHelper->EnablePhyTraces ();
    lteHelper->EnableMacTraces ();
    lteHelper->EnableRlcTraces ();
    lteHelper->EnablePdcpTraces ();
    Ptr<RadioBearerStatsCalculator> rlcStats = lteHelper->GetRlcStats ();
    rlcStats->SetAttribute ("EpochDuration", TimeValue (Seconds (1.0)));
    Ptr<RadioBearerStatsCalculator> pdcpStats = lteHelper->GetPdcpStats ();
    pdcpStats->SetAttribute ("EpochDuration", TimeValue (Seconds (1.0)));
    //连接自定义跟踪接收器，用于 RRC 连接建立和切换通知
    Config::Connect ("/NodeList/*/DeviceList/*/LteEnbRrc/ConnectionEstablished",
                     MakeCallback (&NotifyConnectionEstablishedEnb));
    Config::Connect ("/NodeList/*/DeviceList/*/LteUeRrc/ConnectionEstablished",
                     MakeCallback (&NotifyConnectionEstablishedUe));
```

```
    Config::Connect ("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverStart",
                     MakeCallback (&NotifyHandoverStartEnb));
    Config::Connect ("/NodeList/*/DeviceList/*/LteUeRrc/HandoverStart",
                     MakeCallback (&NotifyHandoverStartUe));
    Config::Connect ("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverEndOk",
                     MakeCallback (&NotifyHandoverEndOkEnb));
    Config::Connect ("/NodeList/*/DeviceList/*/LteUeRrc/HandoverEndOk",
                     MakeCallback (&NotifyHandoverEndOkUe));
    //连接额外的跟踪以进行更多的实验跟踪
    Config::Connect
("/NodeList/4/DeviceList/*/ComponentCarrierMapUe/*/LteUePhy/ReportUeMeasurements",  MakeCallback
(&NotifyUeMeasurements));
    Config::Connect                    ("/NodeList/4/ApplicationList/*/$ns3::PacketSink/Rx",MakeCallback
(&NotifyPacketSinkRx));
    Config::Connect
("/NodeList/*/DeviceList/*/$ns3::LteEnbNetDevice/ComponentCarrierMap/*/FfMacScheduler/$ns3::RrFfMa
cScheduler/WidebandCqiReport", MakeCallback (&NotifyCqiReport));
    //延迟跟踪连接，直到 TCP 套接字存在
    Simulator::Schedule (Seconds (1.001), &ConnectTcpTrace);
    //启动位置追踪
    Simulator::Schedule (Seconds (0), &TracePosition, ueNodes.Get(0), positionTracingInterval);
    //开始执行程序
    Vector vUe = ueNodes.Get (0)->GetObject<MobilityModel> ()->GetPosition ();
    Vector vEnb1 = enbNodes.Get (0)->GetObject<MobilityModel> ()->GetPosition ();
    Vector vEnb2 = enbNodes.Get (1)->GetObject<MobilityModel> ()->GetPosition ();
    std::cout << "Initial positions:   UE: (" << vUe.x << "," << vUe.y << "), "
              << "eNB1: (" << vEnb1.x << "," << vEnb1.y << "), "
              << "eNB2: (" << vEnb2.x << "," << vEnb2.y << ")" << std::endl;
    std::cout << "Simulation time: " << simTime << " sec" << std::endl;
    Simulator::Schedule (reportingInterval, &ReportProgress, reportingInterval);
    Simulator::Stop (Seconds (simTime));
    Simulator::Run ();
    Simulator::Destroy ();
    //关闭文件描述符
    g_ueMeasurements.close ();
    g_cqiTrace.close ();
    g_packetSinkRx.close ();
    g_tcpCongStateTrace.close ();
    g_positionTrace.close ();
    return 0;
}
```

根据上述实验代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/tools/**

ns3/handover/scratch”命令切换路径，接着使用“**touch lte-tcp-x2-handover.cc**”创建实验代码文件，将上述代码抄录至该文件中；

②在命令行终端运行“**./waf --run "lte-tcp-x2-handover --speed=20 --x2Distance=500 --yDistanceForUe=1000 --useRlcUm=0 --handoverType=A2A4"**”执行实验代码（handoverType、speed、useRlcUm 等可根据所完成的实验目标修改[8]，读者可自行尝试），运行成功后会在“**handover**”中产生 **lte-tcp-x2-handover.tcp-receive.dat、lte-tcp-x2-handover.tcp-state.dat** 等数据文件。

7.3.3 实验结果分析

在前面实验操作②生成的数据基础上，可进行结果可视化的分析。在该实验中，主要分析 TCP 吞吐量、RSRQ。其中，吞吐量数据可视化的 Python 脚本如下：

```
import sys
import matplotlib
matplotlib.use('agg')
import matplotlib.pyplot as plt
import argparse
parser = argparse.ArgumentParser()
# Positional arguments
parser.add_argument("--fileName", help="file name", default="../lte-tcp-x2-handover.tcp-receive.dat")
parser.add_argument("--plotName", help="plot name", default="lte-tcp-x2-handover.tcp-throughput.pdf")
# Optional timestep argument
parser.add_argument("--timestep", help="timestep resolution in seconds (default 0.1)")
parser.add_argument("--title", help="title string", default="Lte Handover TCP throughput")
args = parser.parse_args()
timestep = 0.1
if args.timestep is not None:
    timestep = float(args.timestep)
# create histogram of bits per timestep from input file
times=[]
bits_per_timestep=[]
fd = open(args.fileName, 'r')
current_time = 0
current_bits = 0
for line in fd:
    l = line.split()
    if line.startswith("#"):
```

[8] https://wp.ece.uw.edu/wp-content/uploads/sites/36/2019/04/Experiment_3_LTE.pdf

```
            continue
        timestamp = float(l[0])
        if (timestamp < (current_time + timestep)):
            current_bits = current_bits + int(l[1])*8
        else:
            times.append(current_time + timestep)
            bits_per_timestep.append(current_bits)
            current_time = current_time + timestep
            while (current_time + timestep <= timestamp):
                times.append(current_time + timestep)
                bits_per_timestep.append(0)
                current_time = current_time + timestep
            current_bits = int(l[1])*8
# finish last sample
times.append(current_time + timestep)
bits_per_timestep.append(current_bits)
fd.close()
if len(times) == 0:
    print("No data points found, exiting...")
    sys.exit(1)
# Convert observed bits per timestep to Mb/s and plot
rate_in_mbps = [float(x/timestep)/1e6 for x in bits_per_timestep]
plt.plot(times, rate_in_mbps)
plt.xlabel('Time (s)')
plt.ylabel('Rate (Mb/s)')
plt.ylim([0,20])
plt.title(args.title)
#plt.show()
plotname = args.plotName
plt.savefig(plotname, format='pdf')
plt.close()
sys.exit (0)
```

将上述代码保存为“**throughput.py**”文件，然后直接执行“**python throughput.py**”，可得到如下图所示的结果。从结果可知，基站切换时会造成 TCP 吞吐量降低，然后逐步恢复。

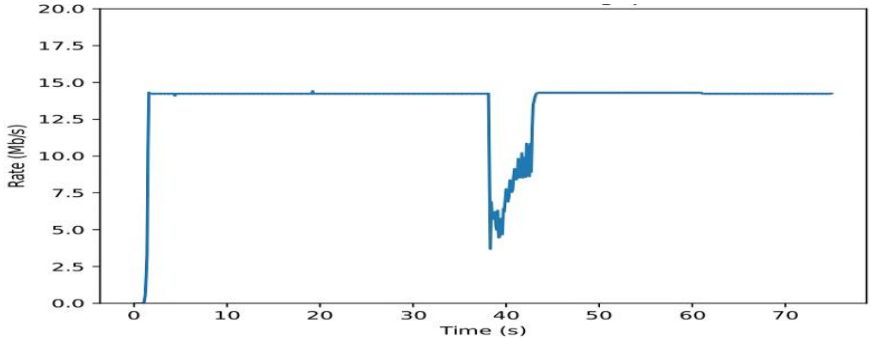


图 7.7　LTE 基站切换时 TCP 吞吐量变化

RSRQ 数据可视化的 Python 脚本如下：

```
import sys
import matplotlib
matplotlib.use('agg')
import matplotlib.pyplot as plt
import argparse
parser = argparse.ArgumentParser()
# Positional arguments
parser.add_argument("--fileName", help="file name", default="../lte-tcp-x2-handover.ue-measurements.dat")
parser.add_argument("--plotName", help="plot name", default="lte-tcp-x2-handover.rsrq.pdf")
# Optional timestep argument
parser.add_argument("--title", help="title string", default="LTE handover RSRQ")
args = parser.parse_args()
times1=[]
times2=[]
values1=[]
values2=[]
fd = open(args.fileName, 'r')
for line in fd:
    l = line.split()
    if line.startswith("#"):
        continue
    if l[1] == "1":
        times1.append(float(l[0]))
        values1.append(float(l[4]))
    elif l[1] == "2":
        times2.append(float(l[0]))
        values2.append(float(l[4]))
fd.close()
if len(times1) == 0:
    print("No data points found, exiting...")
    sys.exit(1)
plt.scatter(times1, values1, marker='.', label='cell 1', color='red')
if len(times2) != 0:
    plt.scatter(times2, values2, marker='.', label='cell 2', color='blue')
plt.xlabel('Time (s)')
plt.ylabel('RSRQ (dB)')
plt.title(args.title)
#plt.show()
plotname = args.plotName
plt.savefig(plotname, format='pdf')
plt.close()
sys.exit (0)
```

将上述代码保存为“**rsrq.py**”文件，然后直接执行“**python rsrq.py**”，可得到如下的输出：

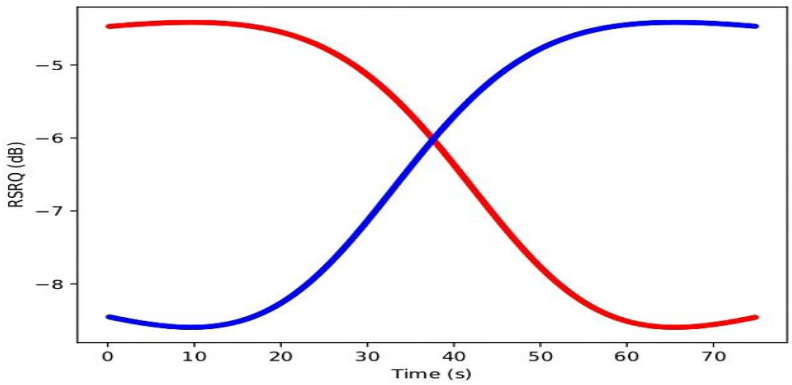


图 7.8　LTE 基站切换时 RSRQ 变化

7.4 扩展实验*

本章介绍了蜂窝（4G LTE）无线广域网的基本概念和结构，并用 NS-3 分析了基站切换时 LTE 网络的 TCP 传输性能。在此基础上，读者可深入探索，扩展实验，具体如下：

（1）将实验中的 speed 设置为 0，并变化 yDistanceForUe 的值，分析 RSRQ 的变化；

（2）变化实验中的 speed 设置，分析 TCP 吞吐量的变化；

（3）变化 yDistanceForUe 设置，分析 TCP 吞吐量的变化；

参考文献

[1] H. Poddar, T. Yoshimura, M. Pagin, et al. Full-Stack End-To-End mmWave Simulations Using 3GPP and NYUSIM Channel Model in ns-3. In Proc. ICC, May 2023, Rome, Italy.

[2] M. Mezzavilla, M. Zhang, M. Polese, et al. End-to-End Simulation of 5G mmWave Networks. IEEE Communications Surveys & Tutorials, 2018, 20(3): 2237-2263.

[3] B. H. Kim, D. Calin. On the Split-TCP Performance over Real 4G LTE and 3G Wireless Networks. IEEE Communications Magazine, 2017, 55(4): 124-131.

[4] B. Nguyen, A. Banerjee, V. Gopalakrishnan, et al. Towards Understanding TCP Performance on LTE/EPC Mobile Networks. In Proc. AllThingsCellular, Aug. 2014, New York, United States.

[5] Y. Ren, W. Yang, X. Zhou, et al. A Survey on TCP over mmWave. Computer Communications, 2021, 171: 80-88.

[6] M. Zhang, M. Polese, M. Mezzavilla, et al. Will TCP Work in mmWave 5G Cellular Networks? IEEE Communications Magazine, 2019, 57(1): 65-71.

[7] M. Mezzavilla, S. Dutta, M. Zhang, et al. 5G MmWave Module for the ns-3 Network Simulator.

In Proc. MSWiM, Nov. 2015, Cancun, Mexico.

[8] 赵亚军, 郁光辉, 徐汉. 6G 移动通信网络: 愿景、挑战与关键技术. 中国科学: 信息科学, 2019, 49(8): 963-987.